

# COS 710 Assignment 1

Jacobus Smit u21489476

---

## Environment setup

For the current research an initial population size of 200 has been selected to maintain computational efficiency and a good representation of the search space.

- **Search Space Coverage:** With a population of 200 individuals ensured that there is enough diversity to allow the GP to experiment and explore different combinations of terminals {L1, L2, L3, L4} and function set {+, -, \*, /}.
- Duplicates in the initial population were permitted and not explicitly checked since in a population with a max depth of 6 together with the Grow population generation method, provided a statistically negligible change for a duplicate tree to be generated.
- To ensure that selection pressure remains without having the selection process be purely random, a tournament selection size of 4 was selected.
- A crossover probability of 70% was used to ensure that the chance for exploitation is present while still having the probability of a stable convergence without breaking a good working tree structure.
- Since crossover is an exploitative operator, there was a need to maintain genetic diversity which would then be introduced with a mutation probability of 10%.
- Point mutation was used with the depth being randomly generated, and the algorithm going on a random walk down the children until the desired depth was reached. If the depth was reached there was a 50% chance to be replaced by either a function or a terminal. However, if a terminal was reached mid-walk, it was simply replaced by another random terminal.
- To ensure that at least the best of the best always progress to the next generation, an elitism count of 10 was introduced which still avoided early convergence while keeping the strongest of the population.
- To allow growing of the trees, a 70% chance of pruning has been selected to promote exploration of the search space. If pruning was selected, a tree would only be pruned if the depth reached  $D + 2$  (8 levels) in order to test if a greater representation of the search space was required.
- Lastly, a training/testing ratio of 80/20 was selected as is the industry standard.

## Machine Technical Specifications

The project was developed on 2 separate machines with github used as a central repository to share across the 2 machines. The first machine that was used to complete the assignment was a MacBook Pro, while the secondary machine used to complete the assignment was a Windows machine, thus ensuring that the program does work on not only multiple machines, but also multiple Java versions.

| Component    | Environment A (macOS) | Environment B (Windows) |
|--------------|-----------------------|-------------------------|
| Processor    | Apple M3 Pro          | AMD Ryzen 5 3600 6 Core |
| Ram          | 18GB                  | 16GB                    |
| OS           | macOS                 | Windows 11              |
| Java Version | Java 23.0.2           | Java 17                 |
| IDE          | IntelliJ              | IntelliJ                |

## Description of the dataset

The dataset consists of electricity load measurements recorded at 15 minute intervals. Since the data is already between 0 - 1, there was no need to normalise it before beginning to process the data.

## Description of the performance metric used to assess the quality of the predictor

To evaluate the accuracy of the GP, I have decided to use MSE (Mean Squared Error) since MSE applies heavier penalties to large outliers, however since the input data ranges between 0 and 1, it is expected for MSE to get extremely small but also extremely precise in value and as is shown in the results. This allows more realistic comparison of trees across multiple generations.

## Description of the representation used and the terminal and function sets

- Based on the data set I have decided to break down the terminals into a set of size 4 namely {L1, L2, L3, L4}. Each terminal would represent a different time within the window which I have decided to look at. Since the data was broken down into 15 minute periods, these 4 terminals would make up an entire hour (T-4, T-3, T-2, T-1) to predict the next value in the series. The GP would make a single step ahead forecast to ensure maximum accuracy for the next interval.
- The function set was broken down into another set of size 4 namely {+, -, \*, /}. To maintain population stability two constraints were introduced, namely Arithmetic Penalty and Fitness Capping.
- The Arithmetic Penalty was introduced for protected division meaning that in the event of an illegal operation like division by 0, in which case the tree would be assigned a static fitness of 1000.
- Secondly, the Fitness Capping was introduced to combat improbable or divergent MSE value which was often introduced due to the instability of the complex tree structures. They were similarly capped at a fitness of 1000.
- These penalties ensured that while the trees remained in the population, they were effectively excluded from the next generation as they would be deemed unfit when compared to other trees.

## Description of initial population generation

For this research, the initial population generation was done using the Grow method with a maximum depth constraint of 6. This was done since unlike the Full population method, the

Grow method is more exploratory as it allows for branches of different lengths to grow, which in turn gives a different view on the search space than Full would. This allows a fine grained control between flexibility and computational safety. Throughout the tree building process there is a random choice between whether the next node being built is going to be a function or a terminal. Once that has been decided, another random variable is used to decide which element in the selected set will be used.

## Description of the fitness function and fitness evaluation

- Each tree is evaluated with a bottom-up traversal. The algorithm walks the tree and computes a single prediction for each 15-minute data sample.
- When a terminal is reached, the corresponding load value from the current data row is loaded and returned to its parent which is a function.
- For function nodes, the algorithm recursively evaluates all the children first after which it will apply the operator and in that case the value is returned to the functions parent until the root is reached and a singular value is returned.
- The final fitness score is the MSE (Mean Squared Error) which is calculated by comparing the tree's prediction against the target load value for every sample in the training set. The final value is the stored in the root node of the tree in order to make fitness retrieval easier when the selection process begins.

## Description of the selection method

In the research two selection methods were used. Firstly, to ensure that the absolute best of each generation was passed onto the next, elitism was used for the first and best 10 trees of the population. Since elitism is highly exploitative and could lead to early convergence, only a small portion (5%) of the population was selected to proceed.

Secondly, was the tournament selection. Since I had a large population a tournament size of 4 was selected as to have a balance between selection pressure but also as to not make it purely random selection.

## Description of genetic operators

In the research two genetic operators were used in order to maximise exploitation while still keeping exploration in play.

Firstly, crossover was used with a 70% chance of execution. This allows for a small portion of the population to progress to the next generation unscathed no matter what their fitness is or their structure is.

Secondly to ensure that there is at least some exploration once the trees have converged I have decided to give the trees a 10% chance to mutate as well. This is done by taking a random depth, going to that depth on a random walk through the children until the desired depth was reached. If the selected node is a terminal, it's then replaced by another terminal. However, if the selected node is a function, there is a 50% chance to be replaced by either a function or a terminal. If it is replaced by a terminal all the children of the selected node are removed, if it is replaced by a function, then the children are copied over in the exact same order to ensure that working subtrees aren't destroyed too much.

## Reiteration of parameter values

| Parameter              | Configuration                        |
|------------------------|--------------------------------------|
| Population Size        | 200                                  |
| Initialization Method  | Grow Method (Max Depth: 6)           |
| Terminal Set           | {L1, L2, L3, L4}                     |
| Function Set           | {+, -, *, /}                         |
| Selection Method       | Tournament Selection (Size $k = 4$ ) |
| Elitism                | Top 10 individuals (5%)              |
| Crossover Probability  | 70%                                  |
| Mutation Probability   | 10% (Point Mutation/Random Walk)     |
| Pruning Strategy       | 70% chance at Depth 8 (D+2)          |
| Training/Testing Split | 80% / 20%                            |

## Results

Summary Statistics Table. The results obtained from the result set indicate a high degree of algorithmic stability. The table below provides the aggregate performance metrics across 10 independent experimental runs.

| Metric         | Mean (Average)        | Standard Deviation ( $\sigma$ ) |
|----------------|-----------------------|---------------------------------|
| Final Best MSE | $2.58 \times 10^{-4}$ | $3.57 \times 10^{-5}$           |
| Execution Time | 130.38 seconds        | 45.47 seconds                   |

The low standard deviation in the Final Best MSE ( $3.09 \times 10^{-5}$ ) demonstrates that the Genetic Programming configuration is robust. Despite the stochastic nature of the initialization and genetic operators, the population consistently converged toward a high-precision solution for the 15-minute electricity load data.

### Best MSE Convergence Data

| Metric         | Mean (Average)        | Standard Deviation ( $\sigma$ ) |
|----------------|-----------------------|---------------------------------|
| Final Best MSE | $2.58 \times 10^{-4}$ | $3.57 \times 10^{-5}$           |
| Execution Time | 130.38 seconds        | 45.47 seconds                   |

The graphs below were generated for seed 2.

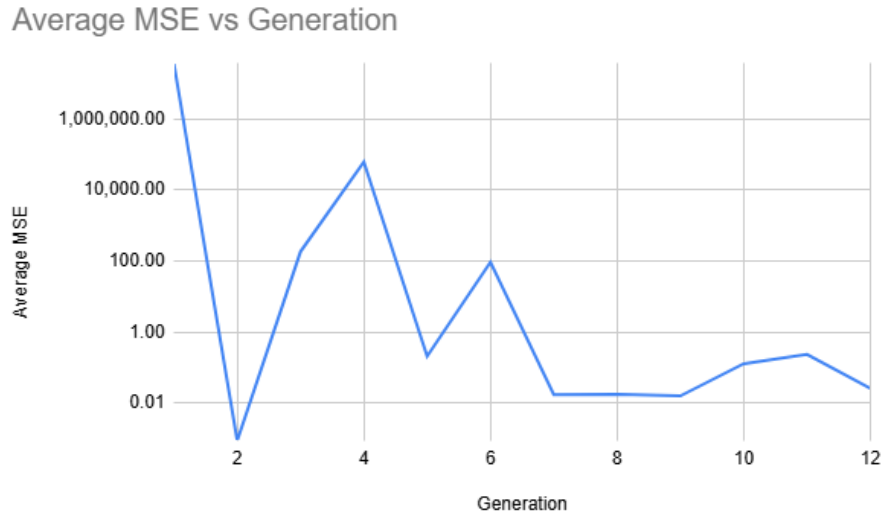


Figure 1: Average population MSE across 12 generations (Seed 2).

The above figure illustrates the average MSE of the entire population across the 12 generations for seed 2. Unlike the best MSE in the next graph, which follows a steady downward trend, the Average MSE exhibits significant fluctuations.

As expected, in generation 1 there is an extremely high MSE value as the trees generated have not been optimized at all. The spikes at 3, 4 and 6 show the result of crossover and mutation being injected into the population, which even though some of them are unfit, is still the result of exploration. As can be seen from population 6 to 12 the trees can be seen converging with slight exploration still occurring around generation 9 to 12.

While the average MSE did display some chaos in the results, the graph below shows the gradual decrease in the MSE which is a result of the best tree slowly but surely converging to the most optimal structure.

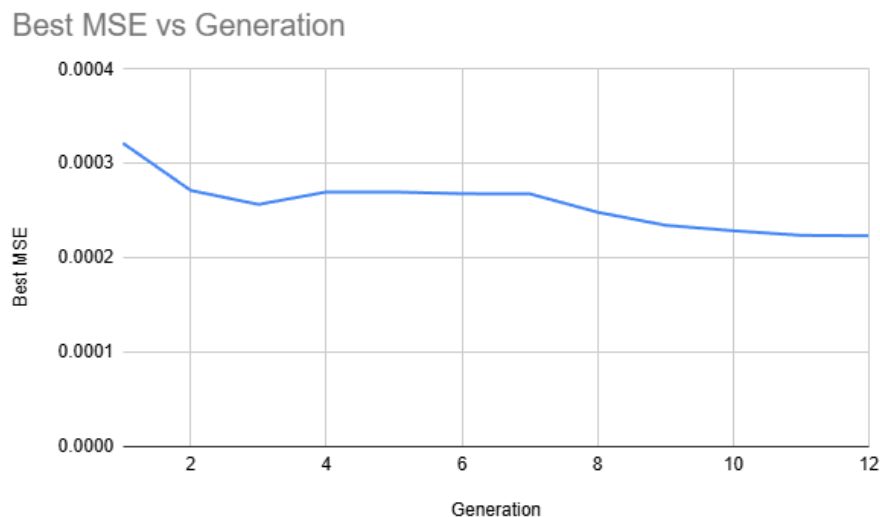


Figure 2: Best tree MSE convergence trend (Seed 2).

The initial drop in MSE is most likely due to the tournament selection taking place, but could also be due to the elitism working together with the mutation and crossover operations to ensure

that the tree does keep on improving.

The plateau in the middle of the generations indicates either that the GP had great difficulty arranging trees in more efficient arrangements. On the contrary, it could also indicate that the GP has reached a local optimum before having a breakthrough after generation 7.

Lastly, the gradual smoothening of the graph indicates the convergence of the tree on a formula that is acceptable and most optimal given the training data set.

Below is the MSE achieved along with the time it took to run through the entire data set for training as well as the 12 generations it was running for.

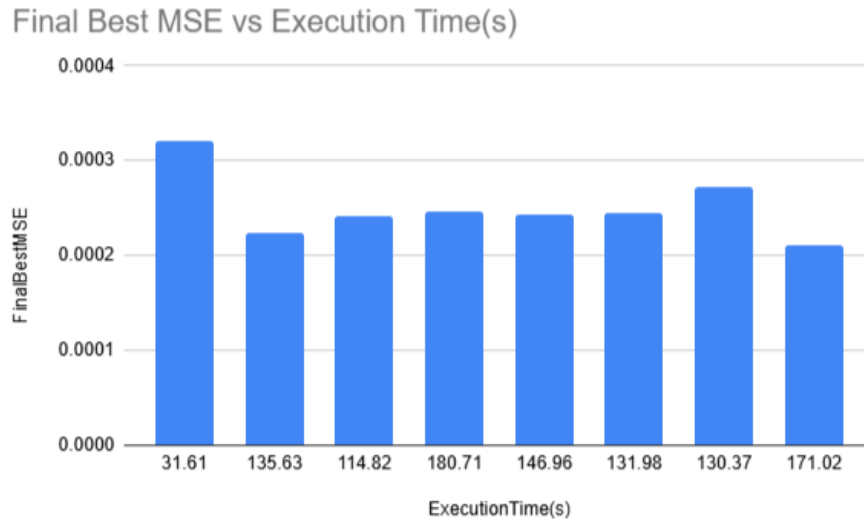


Figure 3: Performance distribution across experimental seeds.

As can be seen, there is a single extreme outlier (seed 1) which completed the entire training process and generation runs in less than a minute. When inspecting these trees, it was observed that the trees converged to a local maximum which consisted of a single terminal (L1) and it failed to break out of this system due to the nature of having an extremely linear tree with no branches or nodes to swap with other trees. This could be improved by not allowing a tree with a depth of only 1, but also allowing the pruning and mutation functions to swap terminals with functions allowing the tree to grow more.

Furthermore, it can be seen that generally, the more complex a tree is the better it scores on the fitness functions, not including the 135.63 second run which is also a bit of an outlier but not the best that has been achieved.

## Master Summary Table

| Seed           | Final Best MSE (Gen 12) | Execution Time (s) |
|----------------|-------------------------|--------------------|
| Seed 1         | $3.2016 \times 10^{-4}$ | 31.61              |
| Seed 2         | $2.2346 \times 10^{-4}$ | 135.63             |
| Seed 3         | $2.4128 \times 10^{-4}$ | 114.82             |
| Seed 4         | $2.4726 \times 10^{-4}$ | 180.71             |
| Seed 5         | $2.4340 \times 10^{-4}$ | 146.96             |
| Seed 6         | $2.4565 \times 10^{-4}$ | 131.98             |
| Seed 7         | $2.7218 \times 10^{-4}$ | 130.37             |
| Seed 8         | $2.1162 \times 10^{-4}$ | 171.02             |
| Seed 9         | $2.2688 \times 10^{-4}$ | 141.54             |
| Seed 10        | $2.2716 \times 10^{-4}$ | 132.99             |
| <b>Mean</b>    | $2.3765 \times 10^{-4}$ | 142.89             |
| <b>Std Dev</b> | $1.8153 \times 10^{-5}$ | 20.51              |

*(Note: Mean and Standard Deviation are calculated for Seeds 2–10 to represent the performance of evolved multi-node tree structures, excluding the single-terminal convergence observed in Seed 1.)*

## Conclusion

With the goal to evolve a Genetic Programming model which could predict the Nth step forecast, I have decided to implement the Grow initialization strategy with a set of Genetic Operators namely a 70% chance for crossover and 10% chance for mutation and terminal set L1, L2, L3, and L4.

The result was a stable GP across 9 runs (excluding the single terminal outlier).